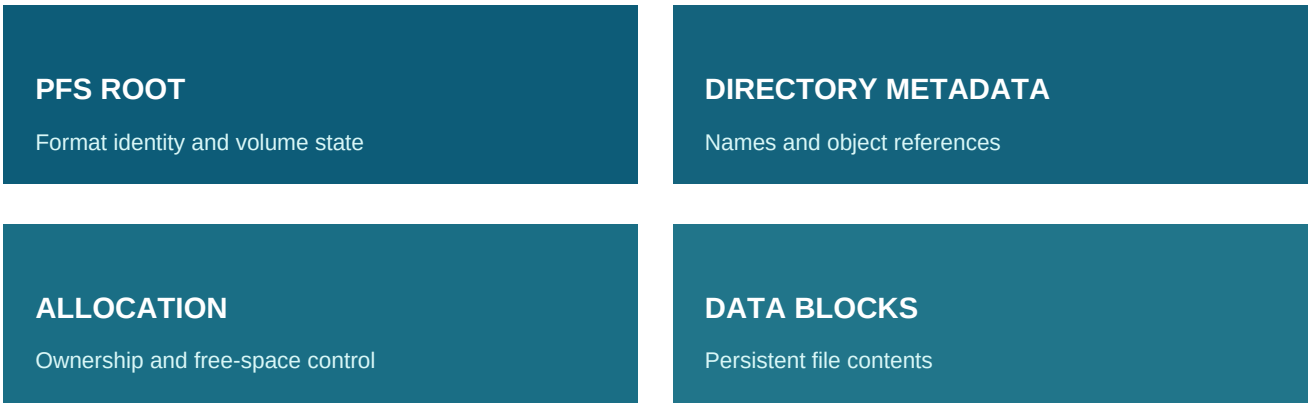
PFS3 - Professional File System 3

OFFICIAL FORMAT / CHRONOLOGY / ENGINEERING

The third-generation Professional File System for Amiga platforms, with its own metadata organisation and performance and recovery priorities.

This guide documents the original format's technical architecture, history and independent source references. InfiltratorOS modules and install packages are implementation artefacts, not the authority for the historic on-disk definition.

CONCEPTUAL MEDIA MAP



CONTENTS

1. Historical chronology
2. Original on-disk architecture
3. Allocation and directories
4. Integrity and recovery
5. Design rationale and limitations
6. InfiltratorOS integration boundary

PFS3 - Professional File System 3

HISTORICAL CHRONOLOGY

Professional File System was developed as a third-party Amiga alternative to original OFS/FFS disk formats. PFS3 continued that line with design priorities such as directory responsiveness, hard disk throughput and recovery following interrupted operations. Its historical commercial and later open-source development should be distinguished from Commodore’s stock Amiga Fast File System.

A preservation and development milestone is the release of the PFS3 All-In-One sources, associated with Michiel Pelt and later contributors. The publicly available source tree provides code-level evidence of allocation, formatting and volume functions. The historical format is not accurately documented by treating PFS3 as a variant of generic Unix inodes or FAT clusters.

PFS3 - Professional File System 3

ORIGINAL ON-DISK ARCHITECTURE

PFS3 stores its own volume control, allocation and directory structures. Its internal design and on-media identifiers are established by the original code, not by the Linux kernel’s AFFS driver. Source files such as allocation.c, volume.c, format.c and struct.h are the appropriate field-level references for interpreting PFS3 volumes. A safe parser must identify the relevant PFS revision and filesystem block size.

File and directory metadata use PFS-specific structures and linkage. The implementation may maintain additional metadata used for efficient allocation and crash recovery. A generic illustration of root, allocation and data areas is a conceptual map only; physical block locations must be taken from format structures and verified source definitions.

PFS3 - Professional File System 3

ALLOCATION AND DIRECTORIES

The allocator manages ownership and free-space structures independently of the legacy OFS/FFS bitmap layout. Directory operations are coordinated with the filesystem's persistent object representation. Large volumes and heavy directory workloads were major practical concerns for Amiga hard-disk users who adopted third-party formats.

Filename, comment, protection and timestamp handling must be matched to the PFS3 source revision and AmigaDOS interface expectations. Cross-platform tools must preserve any unsupported or unknown metadata rather than substituting generic POSIX values. Correct read access does not mean that arbitrary directory mutations can be implemented safely.

PFS3 - Professional File System 3

INTEGRITY AND RECOVERY

PFS3's reputation for resilience stems from its own metadata management and update rules, not from a conventional FAT or JBD-style journal. The format's exact transaction and recovery guarantees must be derived from original source and qualified tests. A source-level description should explain which persistent records select valid filesystem state and how incomplete updates are recognised.

A recovery utility must inspect control records, object references, allocation state and any PFS-specific commit or rollover indicators. When ownership or transaction state is contradictory, read-only recovery and preservation of the original image are safer than selecting a guessed valid state. Claimed self-repair must not be inferred from an unrelated filesystem's design.

PFS3 - Professional File System 3

DESIGN RATIONALE AND LIMITATIONS

PFS3 represents a different evolutionary route from Commodore OFS and FFS, chosen by users who wanted stronger performance and recoverability on Amiga hard disks. Increased sophistication changes the types of corruption and recovery cases an independent driver must understand.

Implementations must respect version-specific capacity, block-size and metadata limits, including PFS3 All-In-One variants. Compatibility with some Amiga operating-system handlers does not prove Linux VFS, UEFI boot or modern device support. A full reference needs ongoing comparison with source revisions and original release notes.

PFS3 - Professional File System 3

INFILTRATOROS INTEGRATION BOUNDARY

PFS3.so is an independent adapter around on-media semantics established by the PFS3 sources. The published .so and optional .ko must be tested against real images from an independent PFS implementation. Defragmentation and recovery claims require physical layout verification and interruption tests.

The public PFS3 source tree and its version history are stronger provenance than unsupported assumptions about the creator’s algorithms. This reference preserves their distinction from the InfiltratorOS release process.

PFS3 - Professional File System 3

PRIMARY SOURCE REFERENCES

These primary technical documents and near-contemporary references provide provenance for the design history and on-disk structures. For exact field sizes or a particular format revision, consult the original specifications and source code, not the conceptual cover illustration.

1. PFS3 All-In-One source and historical version history

https://github.com/tonioni/pfs3aio

2. AmigaOS file system comparison and other contemporary alternatives

https://wiki.amigaos.net/wiki/UserDoc%3AAmigaOS_File_Systems

Our independent .so, optional .ko, build procedures and test evidence describe a separate implementation. The source-level history in this PDF cannot itself qualify production writes, boot, recovery or cross-platform compatibility.

PFS3 / engineering detail

DISK READ PATH / ENGINEERING NOTES

1. Identify both the partition’s RDB DOS type and the boot-block PFS signature; these serve different purposes and may not contain the same identifier bytes.

2. Read the reserved metadata area near the beginning of the PFS volume, rather than searching for an OFS-style midpoint root block.

3. Reconstruct the root and reserved-space bitmap before following anode index blocks. Their pointer fields describe ownership of filesystem metadata.

4. An anode describes a contiguous data extent and may link to another anode. A file’s block map is reconstructed from the anode chain, not a FAT entry per cluster.

5. Parse directory blocks and their variable-length name records using the exact disk format revision. Directory entries refer to anode identities for file contents.

6. Different PFS generations and device addressing methods have different on-disk constraints; interpret versions, physical sector size and reserved block size before decoding pointers.

7. Confirm the extent’s start, length and successor are valid against volume boundaries and that file length is coherent with the extent chain.

8. Separate persistent filesystem state from stale blocks left by past updates. The current root and metadata graph determine which extents are live.

PFS3 / engineering detail

VALIDATION, RECOVERY AND COMPATIBILITY

1. Before a write, check reserved bitmap, anode allocation, directory block ownership and user-data extents as one consistent graph.

2. An anode chain with a loop, an out-of-range block address or a duplicate live extent must not be trusted for writable mounting.

3. Protect the distinction between filesystem metadata blocks and user data. Treating an anode index as file content can destroy both.

4. Preserve the original boot and root state when investigating interrupted operations; stale previous metadata may hold recoverable evidence.

5. Do not confuse PFS3 partition DOS types with file content signatures or with the earlier PFS floppy layout.

6. An implementation claiming TD64 or DirectSCSI support must demonstrate correct addressing beyond legacy device boundaries; a successful small-image test is insufficient.

7. Use the exact pfs3aio source revision for low-level field sizes and byte offsets. This appendix is interpretive and not a substitute for the source structures.

8. Recovery should report conflicting anode and bitmap evidence rather than silently manufacturing a new file topology.